En una sesión de 2 horas, cada grupo amplía el ejemplo de la guía de desarrollo hasta obtener un sistema multiagente de dos agentes que se saludan entre sí. Cada agente se despierta con un estado de ánimo, contento o gruñón, y saluda de forma distinta según ese ánimo. Al final, el grupo entrega el proyecto, la traza de una ejecución y una hoja con cuatro respuestas.

1. Datos de la sesión

Campo	**Valor**
Materia	Sistemas basados en agentes, 4.º del Grado en Inteligencia Artificial, Universidad de Vigo
Duración	2 horas en el laboratorio
Grupos	De 3 a 6 personas, con al menos 2 ordenadores por grupo
Material	*Perfil AgentSpeak (Jason/JaCaMo). Guía de desarrollo, v1.0* (en adelante, «la guía»): capítulos 2 a 11
Requisitos	JDK 17, la distribución AgentSpeak del IADK e Internet la primera vez que se ejecuta un proyecto (guía 2.1). Conviene comprobar antes de la clase que ./gradlew run funciona en el laboratorio
Entrega	Al terminar la clase, en Moovi: un ZIP con el proyecto, la traza y la hoja de respuestas

2. Qué se construye

El punto de partida es el proyecto *Hello world* del creador de proyectos JaCaMo (guía 2.2). El resultado es un sistema con dos instancias del agente greeter que comparten el artefacto contador:

Elemento	**Diagrama del IADK**	**Qué hace**
Operación turno(N) del artefacto Counter	Environment Model	Incrementa el contador y devuelve el nuevo valor en una sola operación
Plan greet (objetivo inicial start)	Component Diagram	Pide un turno al contador: con turno impar, el agente cree mood(happy); con turno par, mood(grumpy)
Despliegue pareja	Deploy Diagram	Arranca dos instancias de Greeter
Objetivo saludar, plan saludando, tarea enviar_saludo	Agent Model y Tasks and Goals Model	Envía a los demás agentes el mensaje saludo(M); el objetivo se satisface con la creencia saludado
Mensajes saludo(M) y respuesta(M)	Interaction Model	Mensajes tell de Greeter a Greeter
Reglas de respuesta	Component Diagram (código del agente)	Al recibir un saludo, el agente responde con un texto que depende de su propio ánimo
Organización saludo_org	Organization Model	Grupo team, rol saludador (2 agentes), esquema saludo_sch con la misión m_saludar y una norma que obliga al rol a comprometerse con ella

Al terminar el bloque 3, la traza debe contener estas líneas (el orden entre agentes y quién está contento puede variar):

[greeter1] Me despierto happy

[greeter1] I am obliged to commit to m_saludar on saludo_sch... doing so

[greeter2] I am obliged to commit to m_saludar on saludo_sch... doing so

[greeter2] Me despierto grumpy

[greeter1] Hola, greeter2. Que alegria verte!

[greeter2] greeter1 me ha respondido de humor happy

[greeter2] Hmpf... hola, greeter1.

[greeter1] greeter2 me ha respondido de humor grumpy

?

3. Roles dentro del grupo

El grupo trabaja sobre una única especificación, en el ordenador A. El ordenador B sirve para leer la guía, abrir el inspector de mentes (http://127.0.0.1:3272) y redactar la hoja de respuestas. Quien está al teclado del ordenador A cambia en cada bloque, para que todos modelen al menos una vez.

Rol	Tarea	3	4	5	6
Modelador	Al teclado del ordenador A: inserta entidades y relaciones en el diagrama del bloque	1	1	1	1
Guía	Lee en voz alta cada paso y comprueba cada relación antes de seguir	1	1	1	1
Verificador	Genera, ejecuta y compara la traza con la esperada	lo hace el Guía	1	1	1
Secretario	Responde las preguntas en la hoja y anota los errores que aparecen	1	1	1	1
Explorador	En el ordenador B, adelanta la lectura del bloque siguiente o prepara un reto	—	—	1	2

Las columnas 3 a 6 indican cuántas personas desempeñan cada rol según el tamaño del grupo.

4. Desarrollo de la sesión

Un bloque no se da por cerrado hasta que la traza muestra lo indicado en la última columna.

Minutos	Bloque	Guía	La traza debe mostrar
0–10	0. Grupos, roles y proyecto *Hello world*	2.2, 2.3, paso 1	Hello world! y The counter is now 1
10–30	1. Dos agentes y su estado de ánimo	Cap. 5, 10 y 11	Dos líneas Me despierto ..., una con happy y otra con grumpy
30–60	2. Saludos que dependen del ánimo	Cap. 4, 5, 6 y 9	Cada agente saluda al otro con el texto de su ánimo y recibe la respuesta
60–85	3. Workspace y organización	Pasos 6–8, cap. 7 y 8	Dos líneas I am obliged to commit to m_saludar ...
85–100	4. Cambiar el código y subirlo al modelo	Paso 9, cap. 11	El saludo nuevo se conserva tras regenerar
100–110	Retos (opcional) o terminar lo pendiente	Cap. 9, 10 y 12	Según el reto
110–120	Entrega y puesta en común	—	ZIP subido a Moovi

Bloque 0. Arranque (0–10 min)

1. Formad el grupo y repartid los roles del primer bloque.
2. Cread el proyecto saludos con *Modules* > *Tools* > *JaCaMo project creator*, partiendo de *Hello world* (guía 2.2).
3. Ejecutadlo con ./gradlew run en la carpeta del proyecto: el agente imprime Hello world! y The counter is now 1. Detenedlo con Ctrl+C.

Bloque 1. Dos agentes y su estado de ánimo (10–30 min)

Cada agente decide su ánimo según el turno que le da el contador compartido. El turno se pide con una operación nueva que incrementa y devuelve el valor a la vez; la pregunta 1 explica por qué.

1. Abrid src/env/tools/Counter.java y añadid esta operación junto a inc (los artefactos se escriben una vez y pertenecen al desarrollador; guía 2.2):

```
@OPERATION
void turno(OpFeedbackParam<Integer> n) {
  ObsProperty p = getObsProperty("count");
```

?

```
    p.updateValue(p.intValue() + 1);
    n.set(p.intValue());
}
```

2. Ejecutad *Modules* > *Tools* > *AppLinker (AgentSpeak)*: la operación turno aparece en el artefacto Counter del modelo (guía, cap. 11).

3. En el diagrama *AgentSpeak code*, haced doble clic sobre el componente de código del plan greet y sustituid su Code por esta regla completa (guía, cap. 5):

```
+!start : true
   <- turno(N);
      if (N mod 2 == 1) { +mood(happy) } else { +mood(grumpy) };
      ?mood(M);
      .print("Me despierto ", M).
```

4. Añadid un Deploy Diagram con un DeploymentPackage pareja y una unidad DeploymentUnitByType greeters (AgentTypeDeployed: Greeter; NumberInstances: 2), unidos con DefinesDeployment (guía, cap. 10, figura 17).

5. Guardad, generad (*Modules* > *Code Generator* > *Jason/JaCaMo AgentSpeak generator* > *Generate*), revisad la pestaña Logs y ejecutad. Deben aparecer greeter1 y greeter2, uno contento y otro gruñón.

Bloque 2. Saludos que dependen del ánimo (30–60 min)

1. **Agent Model** (diagrama *Greeter agent*). Insertad un Goal saludar y un Plan saludando. Relacionad Greeter con saludando (WFResponsable) y saludando con saludar (GTSatisfies), como en el paso 2 de la guía.

2. **Tasks and Goals Model.** Añadid un diagrama *Saludando plan* con Greeter, saludando y saludar. Insertad una Task enviar_saludo y un FrameFact saludado. Relacionad saludando con enviar_saludo (WFContainsTask) (guía, paso 3).

3. **Código de la tarea.** En *AgentSpeak code*, añadid enviar_saludo, insertad un AslCodeComponent enviar_saludo_code unido con CDUsesCode y escribid su cuerpo (guía, paso 4):

```
?mood(M);
.broadcast(tell, saludo(M));
.wait({+saludado})
```

4. **Condición de satisfacción.** En la GTSatisfies de saludando a saludar, marcad la creencia saludado en sus SatisfactionBeliefs (guía 6.1): el objetivo termina cuando el agente sabe que le han respondido.

5. **Interaction Model.** Añadid uno con dos UMLSendsMessage de Greeter a Greeter, ambos con SpeechAct tell: uno con Message saludo(M) y otro con Message respuesta(M) (guía, cap. 9).

6. **Reglas de respuesta.** En *AgentSpeak code*, añadid Greeter, insertad un AslCodeComponent reglas_greeter y unidlo a Greeter con CDUsesCode. Su Code contiene reglas completas, que van a la sección *Other rules of the agent* del .asl (guía, cap. 5):

```
+saludo(_)[source(S)] : mood(happy)
   <- .print("Hola, ", S, ". Que alegria verte!");
      .send(S, tell, respuesta(happy)).

+saludo(_)[source(S)] : mood(grumpy)
   <- .print("Hmpf... hola, ", S, ".");
      .send(S, tell, respuesta(grumpy)).

+respuesta(M)[source(S)]
   <- .print(S, " me ha respondido de humor ", M);
      +saludado.
```

?

7. Al final de la regla del plan greet (bloque 1), cambiad el punto final por un punto y coma y añadid estas dos líneas. La espera da tiempo a que arranque el otro agente antes de saludarlo:

```
.wait(500);

!saludar.
```

8. Generad y ejecutad. Cada agente saluda al otro con el texto de su ánimo y anuncia la respuesta que recibe. Escribid en el código las tildes y los signos de apertura (¡, ¿) solo si el proyecto compila en UTF-8; si no, omitidlos como en los ejemplos.

Bloque 3. Workspace y organización (60–85 min)

1. **Environment Model.** Insertad el Workspace env y relacionadlo con Counter (EContainsArtifact), como en el paso 6 de la guía.

2. **Organization Model.** Añadid un OrganizationModel *Saludo organisation*. Insertad la Organization saludo_org, el OrganizationGroup team, el Role saludador, el Scheme saludo_sch y la Mission m_saludar, y añadid los existentes saludar y Greeter.

3. Relacionadlos como en el paso 7 de la guía, con estos nombres: saludo_org con team (OHasGroup) y con saludo_sch (OHasWF); team con saludador (OHasMember, con cardinalidad 2); saludo_sch con m_saludar (WFDecomposesWF, con cardinalidad 2); saludo_sch y m_saludar con saludar (GTSatisfies); saludador con m_saludar (WFParticipates, Deontic obligation); Greeter con saludador (WFPlays).

4. Guardad, generad y ejecutad. Los dos agentes se comprometen con m_saludar y la traza coincide con la de la sección 2.

Bloque 4. Cambiar el código y subirlo al modelo (85–100 min)

1. Editad src/agt/greeter.asl y cambiad el texto del saludo contento, entre las marcas //#start_node:reglas_greeter y //#end_node:reglas_greeter. Lo que se escribe fuera de las marcas se pierde al regenerar.

2. Ejecutad y comprobad el saludo nuevo.

3. Ejecutad *Modules* > *Tools* > *Code synchronizer (AgentSpeak)*, pulsad *Apply selected* y guardad la especificación (guía, paso 9).

4. Volved a generar y a ejecutar: el saludo nuevo se conserva porque ya está en el modelo.

5. Retos para los grupos que terminen antes

1. **Cuatro saludadores** (cap. 8 y 10). Arrancad 4 instancias y ajustad las cardinalidades del rol y de la misión. Cada agente debe recibir tres saludos: ¿cuántos están contentos?

2. **Un tercer ánimo** (cap. 5). Cuando el turno es múltiplo de 3, el agente cree mood(sleepy) y saluda con un bostezo. Añadid la regla de respuesta correspondiente.

3. **Contagio emocional** (cap. 6 y 9). Un agente gruñón que recibe respuesta(happy) cambia su ánimo a happy (-+mood(happy)) e imprime que le han alegrado el día.

4. **Preguntar antes de saludar** (cap. 9). Antes de saludar, cada agente pregunta el ánimo del otro con askOne y elige un saludo que dependa de los dos ánimos.

5. **Pregunta a un agente de IA** (cap. 12). Quitad la creencia saludado de las SatisfactionBeliefs y preguntad en la pestaña *Asistente JaCaMo* por qué el objetivo saludar ya no termina. Anotad en la hoja si la respuesta es correcta.

6. Entrega

Antes de salir de clase, cada grupo sube a Moovi un ZIP llamado SBA-G<n>.zip que contenga:

- La carpeta del proyecto (spec/specification.xml, src/, el .jcm y los ficheros Gradle), sin build/ ni .gradle/.
- traza.txt, con la salida de la última ejecución (./gradlew run > traza.txt).
- La hoja de respuestas: los nombres del grupo, quién hizo cada rol en cada bloque y las respuestas a las cuatro preguntas siguientes.

Preguntas de la hoja

1. (Bloque 1) Sustituid turno(N) por inc; ?count(N) y ejecutad varias veces. ¿Qué ocurre a veces con el ánimo de los agentes y por qué? ¿Por qué lo evita turno?

2. (Bloque 2) Cuando llega un saludo, ¿cómo elige Jason cuál de las dos reglas de +saludo ejecuta? ¿Qué pasaría si el agente creyera mood(sleepy) y no hubiera regla para ese ánimo?

?

3. (Bloque 2) ¿Por qué el agente contesta con respuesta(M) y no con otro saludo(M)?

4. (Bloque 3) Poned a 1 la cardinalidad de la mision m_saludar, regenerad y ejecutad varias veces. ¿Cuantos agentes reciben la obligacion? Explicadlo con la especificación normativa de Moise y volved a dejarla en 2.

?